

The \$1.95 BaaS Blueprint

A Coolify-style platform for the tier Coolify cannot reach: cheap cPanel shared hosting. One control plane, single-binary backends, and a repeatable recipe that turns a \$2/month hosting account into auth, database, files, realtime, and deploys for a portfolio of side projects.

ORANGEHOST MICRO AS REFERENCE

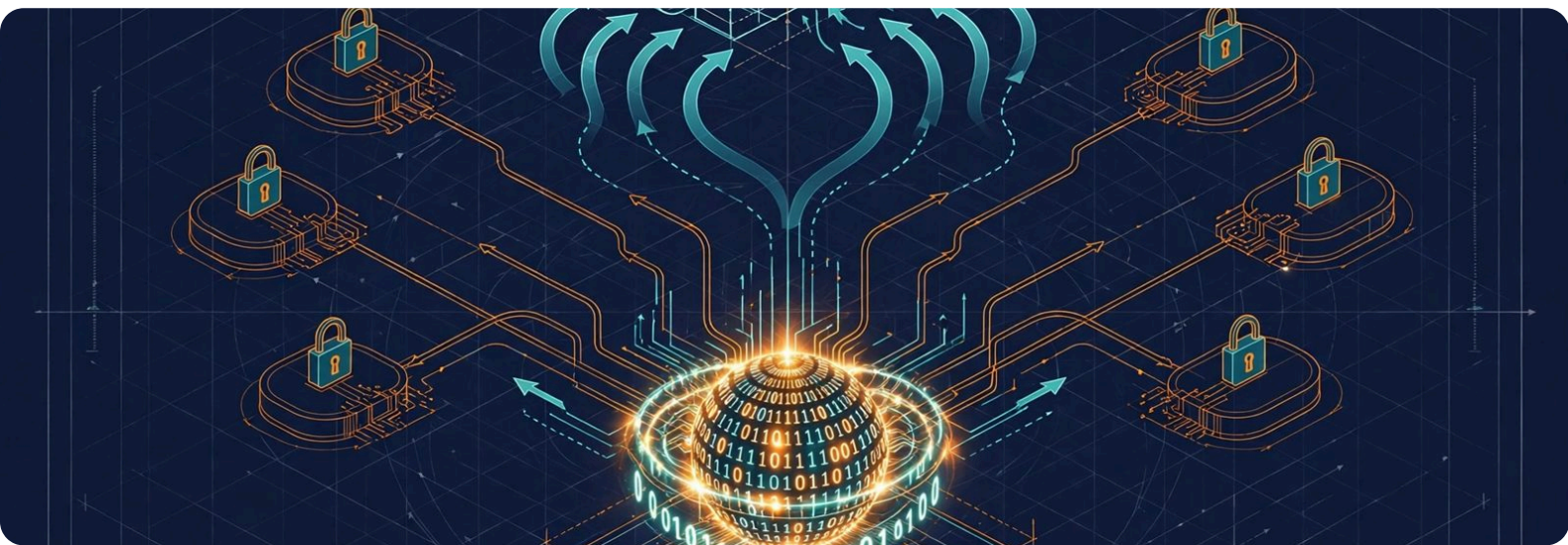
5 GB NVME

1 CPU / 1 GB RAM

POCKETBASE PATTERN

CPANEL UAPI SUBSTRATE

VECTOR / RAG READY



The wedge

Coolify starts at Docker + VPS + 2 GB RAM. Millions of developers start at \$2 shared hosting. Nobody has built the PaaS for that floor.

The trick

Do not fight shared hosting. Use what it already gives you: subdomains, MySQL, cron, SSL, Node selector, and Apache - orchestrated through UAPI.

The payoff

Each new project costs minutes and megabytes, not another \$20/month VPS. Perfect for a 10-20 app portfolio that compounds.

Read this in one sitting, build it in four weeks

Phase 0 cleans the box. Phases 1-2 prove a single PocketBase project end-to-end. Phase 3 turns the manual steps into the product itself: the control plane that other low-budget developers will want to spread.

Reference substrate verified October 5, 2026: OrangeHost Web Hosting-Micro for momsoilchange.com - \$1.95/mo, 5 GB NVMe (2.5 GB used), unlimited MySQL, Node 6-20, Python/Ruby/Git, jailed SSH, no PostgreSQL. Numbers elsewhere are planning estimates and are labelled as such. Host capabilities vary; run the preflight in Section 17 before promising a feature on a new host.

What you are building, in order

- 01 What this is — and what it is not
- 02 Architecture at a glance
- 03 Phase 0: the storage and resource diet
- 04 Shared-hosting constraints you design around
- 05 Running non-root single binaries (PocketBase pattern)
- 06 Ports and reverse proxying through Apache
- 07 Process supervision without systemd
- 08 Multi-project pattern under 1 GB RAM
- 09 SSL and custom domains per project
- 10 Auth, data, and files
- 11 Vector search and AI memory (RAG)
- 12 Backups: Litestream to R2/B2 + restore drills
- 13 Push-to-deploy on shared hosting
- 14 The control plane — the Coolify-style product
- 15 The Reseller Tier — the real multi-tenant substrate
- 16 Limits, failure modes, and the VPS escape hatch
- 17 Four-week MVP roadmap
- 18 Reference kit and preflight checklist

01 What this is — and what it is not

This blueprint builds a **PaaS experience on shared-hosting primitives**. The developer connects a repo, picks a name, and gets a live URL with SSL, a backend with auth and a database, logs, env vars, and backups — the Coolify flow — without ever provisioning a VPS.

It is

- A control plane over cPanel UAPI and WHM API
- A single-binary backend pattern (PocketBase first)
- A playbook for \$2–\$98/month hosting tiers
- Spreadable: any cPanel host, not just OrangeHost

It is not

- Docker on shared hosting (impossible without root)
- A Supabase clone (no Postgres on this tier)
- A Coolify fork (different substrate entirely)
- Production infra for heavy traffic on day one

The thesis in one line

Coolify sells convenience on hardware the user rents by the month. This platform sells the same convenience on hardware the user *already* rents for \$2 — and that is why it spreads: the substrate is already in their pocket.

Who it is for

Indie developers, students, bootcamp grads, and small-shop builders with three to twenty side projects, each too small to justify a \$20 VPS, all of them currently scattered across free-tier dashboards that expire, throttle, or shut down. Your own portfolio — ten to twenty small apps, each with ten to twenty users — is the reference customer.

Architecture at a glance: cPanel is the substrate

Five layers. Only the top one is software you write; the other four are features the host already runs.

1 CONTROL PLANE	Your product. Web UI + API: create project, set env vars, view logs, attach domain, trigger deploy. Stores project metadata in its own PocketBase/MySQL. You build this
2 PROVISIONING	cPanel UAPI (subdomains, MySQL, cron, SSL, Node apps) on a single account; WHM API (create/suspend accounts, packages) on reseller. All scriptable over HTTPS with API tokens. Host provides
3 RUNTIME	One single binary per project — PocketBase (Go) by default; alternatives: Appwrite is too heavy, Supabase needs Docker/Postgres, a small Node app via the cPanel Node selector is the fallback. SQLite on disk, files on disk. PocketBase
4 PROXY / EDGE	Apache + LiteSpeed in front. .htaccess with mod_proxy/mod_rewrite routes <code>project.yourdomain.com</code> to <code>127.0.0.1:<port></code> . AutoSSL terminates HTTPS. WebSocket/SSE caveats in Section 06.
5 DATA / BACKUP	SQLite files per project, replicated continuously by Litestream to Cloudflare R2 or Backblaze B2 free tiers. MySQL available when a workload truly needs it (Section 10). Vector embeddings live beside the data — <code>sqlite-vec</code> by default, MySQL options in Section 11.

Request flow

```

Browser → AutoSSL (443) → Apache vhost (subdomain)
  → .htaccess ProxyPass → 127.0.0.1:8091 (PocketBase)
  → SQLite (WAL) + pb_data/storage (uploaded files)
  → Litestream → R2/B2 (offsite, continuous)
  
```

Why PocketBase first

One static Go binary (~25 MB) delivers authentication (email/password, OAuth), collections as REST + realtime API, file storage, an admin dashboard, and cron hooks — roughly 80% of Supabase's surface with none of its moving parts. It idles at 30–60 MB RAM in planning estimates, which is the only reason the 1 GB tier works at all. If a host kills arbitrary binaries, the same architecture falls back to a Node app under Passenger (Section 05).

Free the box first: the storage and resource diet

Your reference Micro account sits at 2.5 of 5 GB. Target: under 1.5 GB before the first project lands. Every gigabyte freed is headroom a project will not have to fight for.

Where the weight usually is

Culprit	How to find it	Diet move
Error/access logs	<code>du -sh ~/logs ~/public_html/error_log</code>	Truncate, then cap (below)
Old backups in home dir	<code>ls -lh ~/backup* *.tar.gz</code>	Download once, delete from box; backups belong offsite
node_modules in deployed apps	<code>du -sh ~/public_html/*/node_modules</code>	Build locally/CI, deploy artifacts only
Email mailboxes	cPanel → Email → Disk Usage	Forward-and-delete or archive; mail is not free storage
Staging copies / .git	<code>du -sh ~/public_html/*/.git</code>	One canonical deploy dir per project
cPanel trash + tmp	<code>du -sh ~/.trash ~/tmp</code>	Empty monthly via cron

Caps that stay on forever

```
# Log rotation without root: weekly cron, keep 7 days, max 10 MB
find ~/logs ~/apps/*/logs -name "*.log" -size +10M -mtime +7 -delete
# SQLite hygiene per project (in the watchdog, Section 07)
sqlite3 pb_data/data.db "PRAGMA wal_checkpoint(TRUNCATE); VACUUM;"
# PocketBase log cap: --logMaxSizeMB is not a flag; rotate via cron above
# Trash + tmp sweep
find ~/.trash ~/tmp -type f -mtime +14 -delete
```

WAL discipline

PocketBase/SQLite in WAL mode creates a `-wal` file that grows with write bursts. The checkpoint above, run nightly, keeps it bounded. Never copy a live database without its WAL; Litestream (Section 12) handles this correctly — manual `cp` of `data.db` alone does not.

Exit criteria for Phase 0

Disk under 30% used • log rotation cron installed • one offsite backup of the current account downloaded • inventory of everything currently hosted written down (if it is not inventoried, it is not migrated — it is deleted deliberately or kept deliberately).

Design around the cage: shared-hosting constraints

Constraint	Reality on cPanel shared	Design answer
No root, no systemd	You cannot install services	Cron watchdog + start scripts (Sec. 07)
No Docker	Hard no, not a maybe	Single binaries + Node selector (Sec. 05)
CloudLinux LVE limits	CPU/RAM/IO/entry-process caps per account; hitting them = 508 errors	Memory budget (Sec. 08); reseller isolates per tenant (Sec. 14)
No Postgres (this tier)	Micro lists MySQL only	SQLite primary; MySQL when justified (Sec. 10)
Process killers	Some hosts reap long-running background processes	Preflight test (Sec. 17); Passenger fallback
Shared IP / Apache front	You do not bind 80/443	High ports + .htaccess proxy (Sec. 06)
Cron granularity	Often 1-minute minimum, sometimes 15	Watchdog interval = host minimum; design for it
Outbound restrictions	SMTP often relay-only; some ports blocked	Transactional email via API (Resend-style), backups via HTTPS to R2/B2

The mindset shift

On a VPS you ask "what can I install?" On shared hosting you ask "what is already running that I can orchestrate?" Apache, MySQL, cron, SSL issuance, DNS, and the Node runtime are all managed for you — that managed-ness is the product's leverage, not its limitation. Your control plane is an orchestrator of host features, exactly as Coolify is an orchestrator of Docker.

One binary per project: the PocketBase pattern

Install (per project, over jailed SSH)

```
mkdir -p ~/apps/myapp && cd ~/apps/myapp
# Download the Linux amd64 release from pocketbase.io (verify checksum)
./pocketbase serve --http=127.0.0.1:8091 --dir=./pb_data
# Create the admin once: ./pocketbase superuser upsert admin@you.com ***
```

Bind to 127.0.0.1 only. The binary is never exposed directly; Apache is the only public door (Section 06).

What one binary gives each project

Included

- Email/password + OAuth auth
- Collections = tables with REST API
- Realtime subscriptions (SSE-based)
- File storage on disk
- Admin UI at `/_/`
- JS hooks for custom logic

You add

- Per-project subdomain + SSL (Sec. 09)
- Watchdog + log rotation (Sec. 07)
- Litestream replication (Sec. 12)
- Disk quota discipline (folder per project)

When the binary will not survive

Some hosts kill detached processes within minutes. The preflight (Section 17) tests this with a 24-hour sleeper before you build on it. Fallbacks, in order:

- **cPanel Node.js selector / Passenger:** a small Express or Hono app providing auth (JWT) + REST over SQLite/MySQL. Less BaaS surface, fully sanctioned lifecycle — Passenger restarts it and the host supervises it.
- **PHP runtime:** the host's native tongue; a Slim/Laravel-lite API is unglamorous and nearly unkillable on shared hosting.
- **Move that tenant to reseller** (Section 14) where account-level limits are yours to set.

Ports and proxying: Apache is your ingress

Port allocation

Keep a registry — a plain table in the control plane — so projects never collide. Planning pattern on a single account:

Range	Use
8091–8099	PocketBase instances (one per project)
8100–8199	Node/Passenger-adjacent custom apps
8200+	Reserved: control plane internals, Litestream metrics

The .htaccess proxy (subdomain document root)

```
# ~/public_html/myapp/.htaccess (mod_proxy + mod_rewrite hosts)
RewriteEngine On
RewriteCond %{REQUEST_URI} ^/api/.* [OR]
RewriteCond %{REQUEST_URI} ^/_/.*
RewriteRule ^(.*)$ http://127.0.0.1:8091/$1 [P,L,QSA]
# WebSocket/SSE: PocketBase realtime uses SSE over HTTP,
# which survives this proxy; true WebSockets need
# mod_proxy_wstunnel and many shared hosts disable it –
# verify in preflight, degrade to polling if absent.
```

If `mod_proxy` is unavailable (some hosts disable the `[P]` flag), the fallback is cPanel's Node selector for the app itself, or a tiny PHP reverse-proxy shim as a last resort — functional, slower, and a signal to prefer reseller (Section 14) where you control the package features.

Rules that keep you sane

- Never bind a project binary to a public interface; loopback only.
- One subdomain per project, always — path-based multi-tenancy (`/myapp/`) breaks PocketBase asset and API URLs.
- Record port ↔ project ↔ subdomain in one place; the control plane owns this registry.

Supervision without systemd: the cron watchdog

You cannot have systemd. You can have 95% of what you actually used it for: start on boot-equivalent, restart on crash, and a log you can read.

The pattern (per project)

```
~/apps/myapp/
pocketbase      # binary
pb_data/        # data + storage
start.sh        # idempotent launcher (nohup, pidfile)
watchdog.sh     # "if pid dead, run start.sh"
logs/app.log
```

```
#!/bin/sh
# start.sh – safe to run repeatedly
cd "$HOME/apps/myapp"
if [ -f app.pid ] && kill -0 "$(cat app.pid)" 2>/dev/null; then exit 0; fi
nohup ./pocketbase serve --http=127.0.0.1:8091 --dir=./pb_data \
  >> logs/app.log 2>&1 &
echo $! > app.pid
```

```
# crontab – the entire supervisor
* * * * * $HOME/apps/myapp/watchdog.sh
30 3 * * * find $HOME/apps/myapp/logs -name "*.log" -size +10M -delete
```

What this gives up vs systemd — honestly

systemd	Cron watchdog equivalent
Instant restart	Up to one cron interval of downtime (usually 1 min)
Dependency ordering	Start order in watchdog script; keep it simple
journald	Plain log files + rotation (Section 03)
Resource limits	CloudLinux LVE already enforces account limits

For side projects at ten to twenty users, a one-minute worst-case restart is a fair trade for a \$1.95 substrate. Say so in the product copy; do not oversell five-nines on shared hosting.

Many projects, 1 GB: the Micro multi-project pattern

The decision: one shared PocketBase, or one per project?

	One shared instance, many collections	One instance per project
RAM	Lowest (single 30–60 MB)	Scales linearly per project
Isolation	Weak — one crash takes all projects	Strong — blast radius is one project
Backups	All-or-nothing restore	Per-project Litestream streams
Verdict	Only for your own throwaway tools	Default. This is the product pattern

Memory budget (planning estimates — measure in preflight)

Consumer	Estimate
cPanel/Apache/MySQL baseline (host-managed, shared)	Outside your 1 GB app view, but LVE counts your processes
PocketBase idle, per project	30–60 MB
PocketBase under light load	60–120 MB
Litestream, per database	10–20 MB
Control plane (if self-hosted on the box)	40–80 MB — or host it elsewhere (see Sec. 13)
Practical Micro ceiling	5–8 light projects per 1 GB account, with headroom

The rule

Project #9 is not a failure of discipline; it is the signal to graduate to the Reseller Tier (Section 14), where each project stops sharing one 1 GB ceiling. Micro is the proving ground, not the ceiling of the business.

SSL and domains: every project gets a real address

The default (zero marginal cost)

- Control plane calls UAPI to create `project.yourdomain.com` as a subdomain with its own document root.
- AutoSSL issues the certificate automatically; force HTTPS in the project's `.htaccess`.
- The proxy rules from Section 06 live in that document root. Done — the project has a public, encrypted URL.

Custom domains (the feature users pay for)

1. User adds a CNAME: `app.theirclient.com` → `project.yourdomain.com`.
2. Control plane adds it as an addon/parked domain via UAPI and points the document root at the same folder.
3. AutoSSL covers it once DNS resolves; the control plane polls and reports "secure" only when the cert is actually live — no claiming green without evidence.

Gotchas

- AutoSSL issuance can lag DNS by minutes to hours; surface "pending" honestly in the UI.
- Wildcard certs are usually unavailable on shared tiers; per-subdomain AutoSSL is the pattern.
- Keep a registry: `subdomain` ↔ `custom domain` ↔ `project` ↔ `port`. The control plane is the only writer.

Auth, data, and files: what PocketBase already solved

Auth

Email/password and OAuth providers configured per project in the PocketBase admin. Your control plane does not re-implement auth — it provisions the instance and hands the project owner the admin URL.
Rule: the platform never sees end-user passwords.

Data

Collections are tables. SQLite handles thousands of rows comfortably; the failure mode is concurrent-write contention, which side-project traffic rarely reaches. Reach for MySQL (unlimited databases via UAPI) only for write-heavy or reporting workloads, accepting a second data system to back up. Semantic search is its own layer — see Section 11.

Files

Uploads land in `pb_data/storage` on the same 5 GB disk — so file storage is a quota conversation from day one. Per-project soft quotas enforced by a nightly `du` report in the control plane; hard enforcement is "the disk fills and everyone learns," which is why Section 15's per-account isolation matters commercially.

Multi-tenancy inside one project

If a project itself serves multiple end-customers, model tenancy as a `tenant` collection with row-level API rules in PocketBase — application-level tenancy inside a project is a solved pattern. Infrastructure-level tenancy (one customer = one isolated backend) is the Reseller Tier in Section 15. Do not confuse the two layers in product copy.

Vector search and AI memory: RAG on a \$2 box

Every BaaS in 2026 gets asked the same question: “can it do semantic search?” On this stack the answer is yes — without Postgres, without pgvector, and without a second database server. You add a vector collection beside the data you already have.

The position this blueprint takes

Default: `sqlite-vec` inside the project’s existing SQLite file. It fits the architecture you already run — one file per project, Litestream already backs it up, zero extra RAM, zero daemon. **MySQL paths below are the fallback** when the payload already lives in MySQL or the team wants SQL-only. The **hybrid HNSW path is the scale exit** when a single project outgrows brute-force scan. MySQL 9.0 native `VECTOR` is documented for completeness — verify your host actually runs 9.0 before designing on it; most shared hosts run MySQL 8.0 or MariaDB.

What “connect” means when you are done

A project exposes vector search the same way it exposes everything else: over HTTPS through the subdomain you already provisioned. No direct MySQL port is opened to the internet (shared hosting does not give you one you should use, and you would not want it). Frontends and scripts connect in three sanctioned ways:

Consumer	How it connects	Auth
Browser / mobile app	PocketBase REST + a custom hook endpoint, e.g. <code>POST /api/search</code> on <code>project.yourdomain.com</code>	PocketBase user token (Bearer)
Server script / cron job	Same HTTPS endpoint, or direct file access over SSH on the box	Admin token or SSH key
Control plane	Creates the collection, stores embedding-model config as env vars, shows doc/vector counts	UAPI + project admin API

```
// Minimal client shape (any language, same idea)
POST https://myapp.yourdomain.com/api/search
Authorization: Bearer <user-token>
{ "query": "how do refunds work?", "collection": "docs", "k": 5 }
// -> { "results": [{ "id": "...", "content": "...", "distance": 0.21 }] }
```

Embedding generation itself is an outbound HTTPS call (OpenAI, Cohere, Voyage, or a hosted open model) from the PocketBase hook — the shared box never runs a transformer. At 1 GB RAM that is a constraint to design around, not a gap to apologise for: generation is rented, storage and retrieval are owned. Cache embeddings on write; never re-embed unchanged content.

First, check what your MySQL actually is

```
SELECT VERSION(); -- e.g. 8.0.x or 10.x-MariaDB decides which options exist
SHOW VARIABLES LIKE 'version%';
```

Run this in phpMyAdmin before choosing a path. MySQL 9.0+ unlocks Option A. MySQL 8.0 / MariaDB means Option B or C (or `sqlite-vec`, which does not care). MariaDB 10.7+ has its own `VECTOR` type and distance functions on some builds — treat it as “verify on your host,” not as a promise.

Three MySQL paths, one default, one scale exit

Recommended default (not MySQL): sqlite-vec in the project file

```
-- Inside pb_data/data.db — ships with the file Litestream already replicates
CREATE VIRTUAL TABLE vec_docs USING vec0(
  embedding float[1536]
);
-- App code inserts the rowid + vector; payload stays in a normal table
SELECT rowid, distance FROM vec_docs
WHERE embedding MATCH ? AND k = 5 ORDER BY distance;
```

Why this wins on shared hosting: no new server, no stored-procedure permissions to negotiate, works identically on Micro and per-account reseller seats, and restores with the database. Load the extension from the PocketBase process or a small Go/Python sidecar — see the hybrid note below if you outgrow it.

MySQL native and 8.0 paths

Option A — MySQL 9.0+ native VECTOR (when the host truly runs 9.0)

```
CREATE TABLE embeddings (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  document_id VARCHAR(255) NOT NULL,  
  content TEXT,  
  embedding VECTOR(1536) NOT NULL -- match model dims (e.g. 1536)  
);  
INSERT INTO embeddings (document_id, content, embedding) VALUES (  
  'doc_001', 'PocketBase is a single-binary backend solution.',  
  STRING_TO_VECTOR('[0.012, -0.045, 0.321, ...]')  
);  
SELECT document_id, content,  
  DISTANCE(embedding, STRING_TO_VECTOR('[0.015, -0.040, 0.310, ...]'), 'COSINE') AS distance  
FROM embeddings ORDER BY distance ASC LIMIT 5;
```

MySQL 8.0: the shared-hosting workhorse

Option B — MySQL 8.0: JSON storage + pure-SQL cosine (the shared-hosting workhorse)

Exact nearest-neighbour by table scan. Slow, honest, and entirely sufficient for prototypes and small corpora. Store the JSON, create the function once per database:

```
CREATE TABLE embeddings (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  document_id VARCHAR(255) NOT NULL,
  content TEXT,
  vector_json JSON NOT NULL
);

DELIMITER //
CREATE FUNCTION COSINE_DISTANCE(a JSON, b JSON)
RETURNS DOUBLE DETERMINISTIC
BEGIN
  DECLARE i INT DEFAULT 0; DECLARE len INT;
  DECLARE dot_product DOUBLE DEFAULT 0.0;
  DECLARE norm_a DOUBLE DEFAULT 0.0; DECLARE norm_b DOUBLE DEFAULT 0.0;
  DECLARE val_a DOUBLE; DECLARE val_b DOUBLE;
  SET len = JSON_LENGTH(a);
  WHILE i < len DO
    SET val_a = CAST(JSON_EXTRACT(a, CONCAT('$[', i, ']')) AS DOUBLE);
    SET val_b = CAST(JSON_EXTRACT(b, CONCAT('$[', i, ']')) AS DOUBLE);
    SET dot_product = dot_product + (val_a * val_b);
    SET norm_a = norm_a + (val_a * val_a);
    SET norm_b = norm_b + (val_b * val_b);
    SET i = i + 1;
  END WHILE;
  IF norm_a = 0 OR norm_b = 0 THEN RETURN 1.0; END IF;
  RETURN 1.0 - (dot_product / (SQRT(norm_a) * SQRT(norm_b)));
END //
DELIMITER ;

SELECT document_id, content,
  COSINE_DISTANCE(vector_json, '[0.012, -0.045, 0.321, ...]') AS distance
FROM embeddings ORDER BY distance ASC LIMIT 5;
```

Running it on shared hosting

Shared-hosting caveats: stored functions may require `CREATE ROUTINE` privilege — if the host denies it, the same math moves into app code over fetched rows (Option C), no schema change. Always pre-filter (`WHERE tenant / collection`) before scanning.

Control-plane surface for vectors

Per project: embedding model + dimensions (env vars, never hardcoded), collection list with vector counts, a “test query” box calling the project’s own endpoint, and the same backup story as the rest of the data. Dimension changes are a new collection, never an in-place alter.

From scan to index: the scale path

Option C — Normalised vectors in a BLOB + dot product (the fast SQL-adjacent path)

When embeddings are pre-normalised ($|v| = 1$, standard for OpenAI/Cohere output), cosine similarity *is* the dot product: no norms, no division. Pack float32 arrays into a BLOB at the language layer (Go `binary.Write`, Python `struct.pack`), store it beside the payload, and compute the dot product in app code over candidate rows. MySQL stays the durable store; arithmetic happens where it is cheap — the bridge to the hybrid design below.

Strategy	Speed	Comfortable scale	Best for
sqlite-vec (default)	Fast for size	~10k–100k vectors per project	The blueprint's standard project
MySQL 9.0 native VECTOR	Moderate	~100k vectors	Hosts verified on 9.0; simple built-in RAG
MySQL 8.0 JSON + stored proc	Slow (table scan)	< 10k vectors	Prototyping, low-data side projects, SQL-only teams
BLOB + app-layer dot product	Faster than JSON scan	~10k–50k with pre-filtering	Normalised embeddings, MySQL payload already exists
Hybrid: store in DB + in-memory HNSW	Very fast (< 10 ms)	Millions	The scale exit (below)

The scale exit: hybrid architecture (production pattern)

MySQL/SQLite give exact scan, not ANN (HNSW/IVFFlat). When latency stops being acceptable, add an index in front of the data — do not migrate it:

1. **Store** relational data and full payload vectors in MySQL/SQLite (source of truth, backed up as always).
2. **Build** an in-memory HNSW index in the backend process (Go `usearch`, Python `hnswlib`) loaded from the database on boot.
3. **Query** the in-memory index for vector IDs, then hydrate: `SELECT ... WHERE id IN (...)` for content, permissions, and tenant filtering.

On Micro this is a per-project opt-in inside the 1 GB ceiling; on reseller it gets its own LVE envelope. Frontend contract unchanged: same `/api/search` endpoint.

Backups you can restore: Litestream to free object storage

A backup you have never restored is a hope, not a backup. This recipe is continuous, offsite, and costs \$0 at side-project scale.

The recipe

```
# litestream.yml (per project)
dbs:
  - path: /home/USER/apps/myapp/pb_data/data.db
    replicas:
      - type: s3
        bucket: myapp-backups
        path: myapp
        endpoint: https://ACCOUNT_ID.r2.cloudflarestorage.com
        sync-interval: 10s
```

- **Cloudflare R2** free tier or **Backblaze B2** free tier — either works; keys scoped to one bucket per project.
- Litestream ships WAL segments continuously, so restore point objective is seconds, not last night.
- Nightly, also snapshot `pb_data/storage` (uploaded files) with `rclone` to the same bucket — Litestream covers the database only.
- Retain 30 days; test-restore monthly into a scratch subdomain and record the result.

Restore drill (the part everyone skips)

```
litestream restore -o /tmp/restore-test.db s3://myapp-backups/myapp
sqlite3 /tmp/restore-test.db "PRAGMA integrity_check; SELECT count(*) FROM _admins;"
```

Productize it

The control plane shows each project a last-replicated timestamp pulled from Litestream's status — a "backed up 12s ago" line in the dashboard is a trust feature competitors on free tiers do not show. Claim it only from the live timestamp, never from the cron schedule.

Push-to-deploy without a CI runner on the box

The lightweight pipeline

1. **Build elsewhere:** GitHub Actions (free tier) or the developer's machine builds the frontend and any binary artifacts. The shared box never runs a build — it has 1 CPU and better things to do.
2. **Deliver:** webhook or a "Deploy" button in the control plane triggers `git pull` in the project directory over SSH, or unpacks an uploaded artifact.
3. **Swap safely:** for PocketBase upgrades, `stop` → `replace binary` → `start` → `health-check /api/health`; keep the previous binary as `pocketbase.prev` for one-command rollback.
4. **Frontend:** static builds `rsync` into the subdomain document root; the proxy rules stay untouched.

```
# deploy.sh (sketch)
set -e
cd "$HOME/apps/myapp"
curl -sf http://127.0.0.1:8091/api/health || ./start.sh
# ... pull, swap, restart via watchdog, re-check health,
# and roll back to pocketbase.prev if the check fails
```

Git is available on the Micro tier, so `git pull` deploys work today. The control plane's job is orchestration and reporting — commit SHA, timestamp, health result — stored per deploy so "what is live?" is always answerable.

The control plane: the Coolify-style product itself

Everything above was manual. The product is the layer that makes it one click — that is what spreads.

MVP feature set

Feature	Backed by
Create project	UAPI: subdomain + MySQL (optional) + scaffold app dir + allocate port
Deploy	git pull / artifact unpack + health check (Sec. 12)
Env vars	Per-project .env rendered into start.sh
Domains + SSL status	UAPI addon domains + AutoSSL polling (Sec. 09)
Logs	Tail of project log files, rotated per Sec. 03
Backup status	Litestream replica timestamp (Sec. 12)
Vector collections	sqlite-vec default; MySQL 8.0/9.0 options; HNSW scale exit (Sec. 11)
Usage	Nightly du + process RSS report per project
Suspend / resume	Stop watchdog + disable proxy (WHM suspend on reseller)

Where the control plane lives

Not on the \$1.95 box: a small hosted instance drives many accounts over UAPI/WHM, surviving any one account's limits — the migration story in the next section.

Product strategy: moat = experience; API = sanctioned

Wrapping UAPI/WHM is commodity — the defensible asset is feel: **5-minute onboarding** (connect a \$2 account → live auth/DB/storage), a **dashboard that hides every cPanel-ism, one-click vector/RAG**, and **backup/restore non-technical users trust**. Coolify won on feel, not orchestration novelty. Risk is lower than it looks: cPanel & WHM runs a **Developer Portal with versioned API docs (v138), forums, and Discord** — this is a sanctioned integration path. Discipline: pin API versions, watch the changelog, keep raw UAPI behind an adapter (version bump = one-file change).

Positioning: “Coolify for the hosting you already have” — every \$2 cPanel account is a prospect with zero new infrastructure to buy.

The Reseller Tier: the real multi-tenant substrate

Micro mode is MacGyver mode — brilliant for proving the idea, wrong for selling it. The reseller account is where this stops being a hack and becomes a platform: every project or customer gets its own cPanel account, its own resource ceiling, and its own blast radius.

The OrangeHost reseller ladder (as surfaced October 2026)

Plan	Price	cPanel accounts	NVMe	Cost per account
Spark	\$19/mo	25	30 GB	\$0.76
Ignite	\$39/mo	50	60 GB	\$0.78
Radiate	\$59/mo	75	90 GB	\$0.79
Blaze	\$78/mo	100	120 GB	\$0.78
Flare	\$98/mo	125	150 GB	\$0.78

All tiers: unlimited bandwidth, free SSL, email, MySQL, and addon domains. Per-account disk averages ~1.2 GB across the ladder — right-sized for a PocketBase project with disciplined file storage. Verify current pricing and account limits on the host's site before publishing numbers; plan names and prices change.

Why reseller beats one shared account for the platform vision

	Single Micro account	Reseller (Spark+)
Isolation	All projects share one CloudLinux LVE envelope	Each cPanel account gets its own LVE limits — CPU, RAM, IO, entry processes
Noisy neighbor	One runaway project starves every other project	One runaway project hits its own ceiling; neighbors unaffected
Disk	One 5 GB pool, shared fate	Per-account quota; a full disk hurts one tenant
Credentials	Everyone shares one account login model	Each tenant can receive real cPanel access to their own account
Suspension	Hand-edited proxy/watchdog tricks	WHM suspend/unsuspend — billing-grade off switch
Backups	Your Litestream discipline only	Per-account backups + your Litestream layer

The sentence that matters

On Micro, isolation is a convention you enforce. On reseller, isolation is a property of the substrate — CloudLinux enforces it whether your code is awake or not. Sellable multi-tenancy needs the second kind.

WHM API: the provisioning primitive

Reseller unlocks **WHM (Web Host Manager)** and its JSON API — the reseller-tier counterpart to UAPI. This is the API surface your control plane automates. The Coolify-style experience is, at bottom, a friendly UI over these calls:

Platform action	WHM / UAPI primitive
Onboard a customer/project	WHM <code>createacct</code> with a package defining disk, bandwidth, and LVE-backed limits
Define seat tiers	WHM packages (e.g. Starter / Pro project templates)
Change a plan	WHM <code>changepackage</code> / account modification calls
Non-payment or abuse pause	WHM <code>suspendacct</code> / <code>unsuspendacct</code>
Inside the account: subdomain, DB, SSL, cron	Same UAPI calls as Micro mode — code you already wrote
Offboard	WHM <code>removeacct</code> after final backup export

Authenticate with WHM API tokens scoped to the reseller, store them as platform secrets (never per-tenant), and log every provisioning call with its result — your audit trail is what separates a platform from a pile of scripts.

Economics and graduation

Unit economics (planning math, your mileage will vary)

Seat price	Substrate cost (Spark)	Gross margin per seat	25 seats gross
\$9/mo	\$0.76	\$8.24 (92%)	\$206 vs \$19 cost
\$19/mo	\$0.76	\$18.24 (96%)	\$456 vs \$19 cost

Even after payment processing and support, a \$9–19 seat leaves very healthy margin: the substrate is priced for 2005 shared hosting, the experience for 2026 PaaS. That spread is the business — and it makes a real free tier affordable at \$0.76 a seat.

Competitive check: PocketHost (\$9.99/mo per instance)

PocketHost charges **\$9.99/mo per PocketBase instance** (\$59.99/yr, \$149.99 lifetime; 250 MB DB + 10 GB files per slot). Here: ~6 instances on Micro = **~\$0.33 each** (~30x cheaper); Spark = **\$0.76/tenant**. Different product: PocketHost sells managed ops on their infra; this sells the control plane over hosting the customer already pays for, with downloadable SQLite files — the anti-data-hostage position. Tradeoff: no managed ops/global ingress included; Sections 07, 12, and the restore drills close that gap.

Architecture shift and graduation

How the architecture shifts between tiers

Concern	Micro (single account)	Spark+ (reseller)
Tenancy unit	Folder + port + subdomain	Whole cPanel account
Resource ceiling	Shared 1 CPU / 1 GB RAM	Per-account LVE package limits
Runtime	PocketBase instances sharing one account	One PocketBase/BaaS instance per account, same recipe
Proxying	.htaccess port proxy, collision registry	Same proxy pattern inside each account — ports can repeat per account
Backups	Litestream per project	Litestream per account + account-level options
Provisioning API	UAPI only	WHM (accounts/packages) + UAPI (inside accounts)

The migration path: Micro → Spark, one control plane

1. **Start** your own projects on the Micro box (Sections 03–12). Prove the recipe, build the control plane against UAPI.
2. **Graduate:** when a project outgrows shared RAM, earns revenue, or gains a paying customer, the control plane creates its cPanel account on Spark via `createacct`.
3. **Move:** deploy the same recipe in the new account, restore the Litestream stream, repoint DNS, verify health, retire the Micro instance.
4. **Steady state:** Micro is the free tier; Spark accounts are the paid seats. One dashboard throughout — the substrate changed, the product never did.

Why this ordering wins

You validate demand at \$1.95/month, and rent the \$19 tier only when a tenant's growth — not your optimism — asks for it. UAPI works identically inside each reseller account, so the control plane carries across.

Limits, failure modes, and the escape hatch

What breaks first	Symptom	Move
RAM on Micro	508s, killed processes	Graduate tenant to Spark account (Sec. 15)
Disk (uploads)	Quota errors, failed writes	Object storage for media, per-account quotas
SQLite write contention	Lock timeouts under bursts	MySQL for that workload, or VPS tier
WebSocket-dependent app	Realtime silently degrades	SSE/polling, or VPS — do not promise WS on shared
Host kills binaries	Watchdog restart loops	Passenger/PHP fallback; change host or tier
Traffic spike	LVE entry-process caps	Cache aggressively; this is the VPS graduation trigger

The escape hatch is a feature. Because every project is a folder, a SQLite file, and a Litestream stream, exporting a tenant to a VPS (real Coolify, real Docker) is a restore-and-repoint operation. Market it that way: "start at \$9, leave whenever, your data is a file you can download." Customer ownership of data is the anti-hostage pitch, and here it is literally true.

Four weeks from blueprint to first tenant

Week	Build	Done when
1	Phase 0 diet + preflight (Sec. 17) on the Micro account; one manual PocketBase project live on a subdomain with SSL	Health endpoint green, watchdog restart observed
2	Litestream to R2/B2 + restore drill; deploy.sh with rollback; second and third projects to validate the port registry	A restore you performed, logged, with timestamps
3	Control plane MVP: create project, env vars, logs, backup status — driving UAPI	A project created from the UI, not the shell
4	Spark reseller provisioning via WHM; migrate one project Micro → own account; seat pricing page	One tenant isolated in its own cPanel account

Weeks are planning targets, not promises. Each week's "done when" is evidence, not effort — the Trust Stack rule applies to this roadmap too: no evidence, no green.

Reference kit and preflight checklist

Canonical layout (per project, any tier)

```
~/apps/<project>/  pocketbase (+ .prev)  pb_data/ (data.db + storage/)
  start.sh  watchdog.sh  deploy.sh  litestream.yml  logs/app.log
~/public_html/<project-subdomain>/
  .htaccess          # proxy to 127.0.0.1:<port>
  index.html         # frontend build output (if any)
```

Preflight — run before building on any new host

- SSH (jailed) works; `uname -m`, free disk, cron minimum interval recorded
- A test binary survives 24h detached — or Passenger fallback confirmed
- `mod_proxy [P]` flag works from `.htaccess` — or Node selector path confirmed
- AutoSSL issues for a fresh test subdomain; time-to-cert measured
- Outbound HTTPS to R2/B2 works; Litestream test replication restored successfully
- UAPI token created and a subdomain created/deleted via API
- (Reseller) WHM API token works; test account created and removed
- DB version recorded; vector path chosen (sqlite-vec) + 100-vector test timed
- LVE limits read and recorded — the numbers you will budget against

Assumptions this blueprint makes (verify, do not trust)

PocketBase idle memory (30–60 MB), Micro practical ceiling (5–8 light projects), and AutoSSL behavior are planning estimates, not measurements of your account — Week 1 replaces them with observed numbers. OrangeHost plan details as of October 5, 2026. Re-confirm pricing before any public claim.